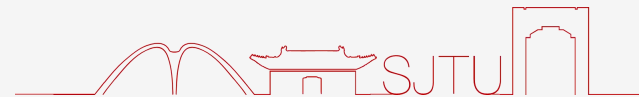




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Lab4 Lock



—— 饮水思源 · 爱国荣校 ——

实验简介

- 重新设计实现Memory allocator和Buffer Cache功能，通过改进数据结构以及锁策略来减少对锁的争用，提高xv6的多核并行能力。

提前阅读

- Book-riscv-rev3.pdf:
 - Chapter 6: Locking
 - Chapter 3, Section 3.5: "Code: Physical memory allocator"
 - Chapter 8, Section 8.1~8.3: "Overview", "Buffer cache layer", and "Code: Buffer cache"
- 阅读代码 [kernel/kalloc.c](#) [kernel/bio.c](#) [kernel/fs.c](#) [kernel/sysfile.c](#) [kernel/file.c](#)

Task1: Memory allocator

实验内容:

- 修改内核的内存分配函数，减少内存申请过程中的对锁的竞争

Xv6 内存管理:

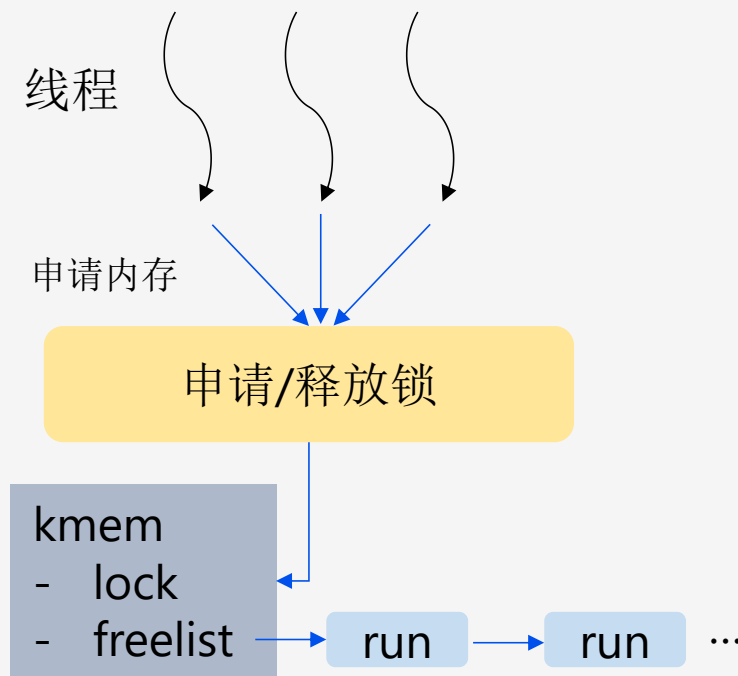
- 物理内存管理器的数据结构
- 获取一个页面

```
struct run {  
    struct run *next;  
};  
  
struct {  
    struct spinlock lock;  
    struct run *freelist;  
} kmem;
```

kernel/kalloc.c

```
kalloc(void)  
{  
    struct run *r;  
  
    acquire(&kmem.lock);  
    r = kmem.freelist;  
    if(r)  
        kmem.freelist = r->next;  
    release(&kmem.lock);  
}
```

kernel/kalloc.c

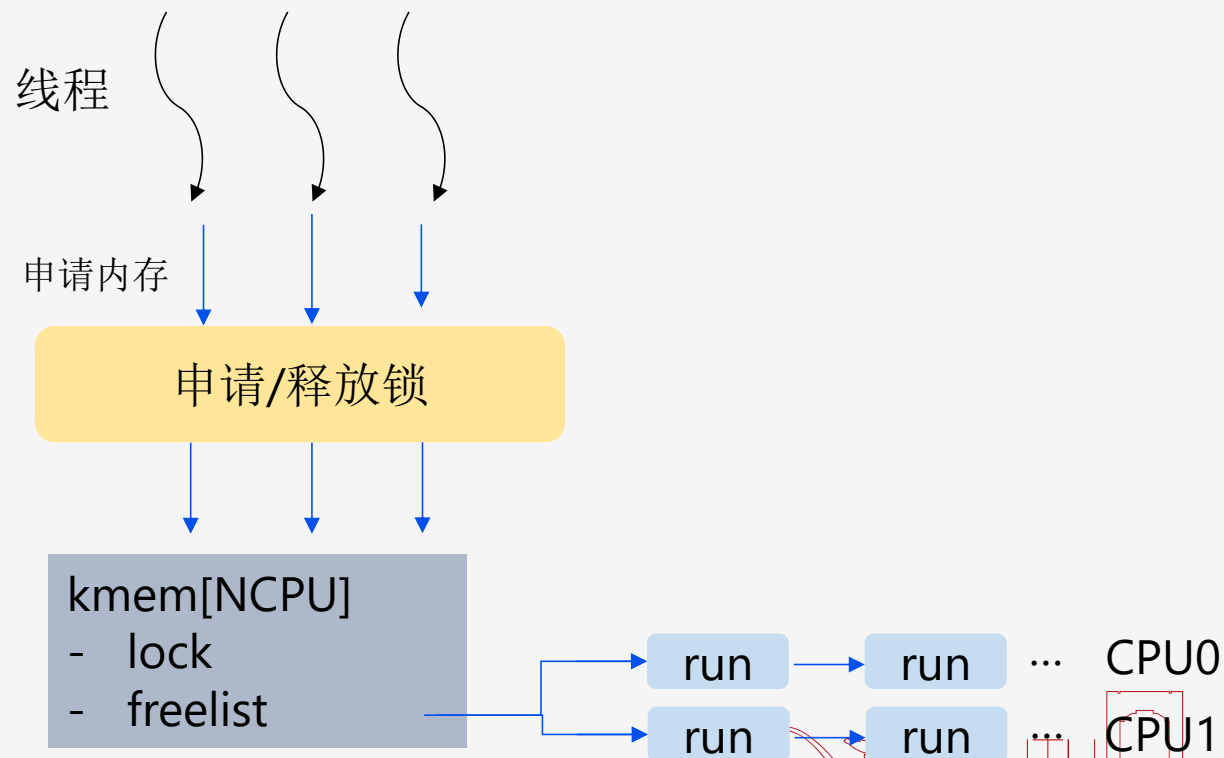
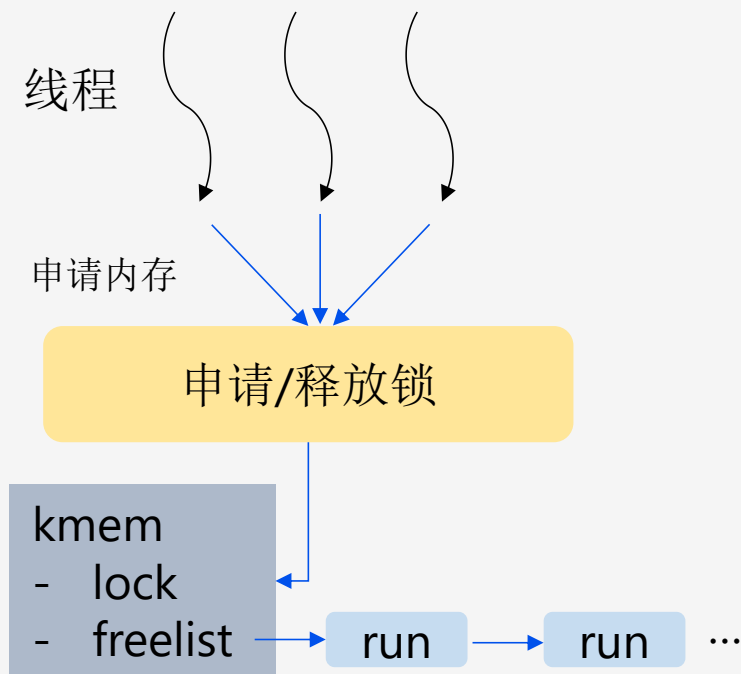


Task1: Memory allocator

实验内容:

- 修改内核的内存分配函数，减少内存申请过程中的对锁的竞争

实现方式: 单freelist -> per-CPU freelist



Task1: Memory allocator



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

实验内容:

- 修改内核的内存分配函数，减少内存申请过程中的对锁的竞争

实现效果: 运行kalloctest, usertests sbrkmuch, usertests -q

```
$ kalloctest
start test1
test1 results:
--- lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 42843
lock: kmem: #test-and-set 0 #acquire() 198674
lock: kmem: #test-and-set 0 #acquire() 191534
lock: bcache: #test-and-set 0 #acquire() 1242
--- top 5 contended locks:
lock: proc: #test-and-set 43861 #acquire() 117281
lock: virtio_disk: #test-and-set 5347 #acquire() 114
lock: proc: #test-and-set 4856 #acquire() 117312
lock: proc: #test-and-set 4168 #acquire() 117316
lock: proc: #test-and-set 2797 #acquire() 117266
tot= 0
test1 OK
```

检测kmem是否有锁竞争

```
start test2
total free number of pages: 32499 (out of 32768)
.....
test2 OK
start test3
child done 1
child done 100000
test3 OK
$ usertests sbrkmuch
usertests starting
test sbrkmuch: OK
ALL TESTS PASSED
$ usertests -q
...
ALL TESTS PASSED
```

检测内存分配是否正常；其他功能是否正常



Task1: Memory allocator



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

实验内容:

- 修改内核的kmem数据结构以及内存分配函数kalloc、kfree等，减少内存申请过程中的对锁的竞争

一些提示:

- NCPU 定义在 kernel/param.h
- 简单起见，freerange函数可以把所有free memory分配到当前CPU的freelist
- cpuid()函数返回当前所在的CPU，需要关闭中断来保证返回结果的正确性
 - 本质：读取tp寄存器中的值，表示当前进程运行在哪个CPU核上
 - 使用push_off() 和 pop_off() 来关闭和开启中断
- 当所在CPU的freelist中没有空闲内存时，需要从其他的CPU窃取空闲内存
 - 窃取需要加锁
 - 窃取半数/任意数量
 - 需要持有多个锁时：所有的代码路径以相同的顺序获取这些锁是很重要的



Task2: buffer cache

实验内容:

- 修改xv6 block cache的实现, 减少内存申请过程中的对锁的竞争

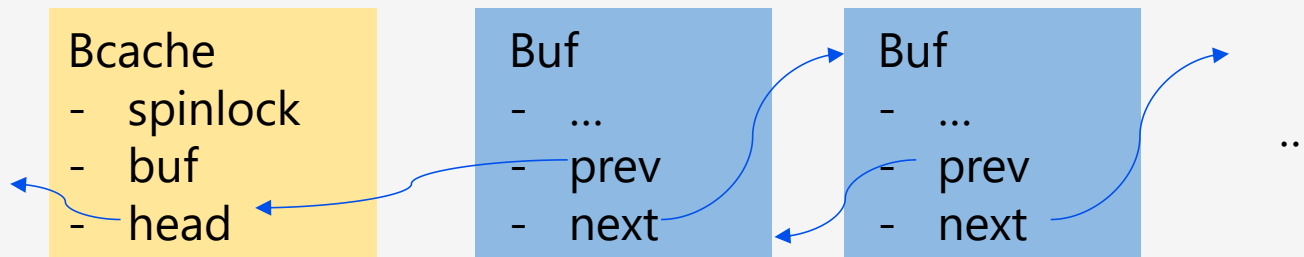
Buffer cache in XV6: kernel/bio.c

```
struct {
    struct spinlock lock;
    struct buf buf[NBUF];

    // Linked list of all buffers, through prev/next.
    // Sorted by how recently the buffer was used.
    // head.next is most recent, head.prev is least.
    struct buf head;
} bcache;
```

```
struct buf {
    int valid; // has data been read from disk?
    int disk; // does disk "own" buf?
    uint dev;
    uint blockno;
    struct sleeplock lock;
    uint refcnt;
    struct buf *prev; // LRU cache list
    struct buf *next;
    uchar data[BSIZE];
};
```

- Buffer cache: 硬盘在内存中的缓冲区
- 为什么要加锁?
 - 多个进程可能同时读写同一个block, 如果没有锁可能会导致链表崩坏
- 全局锁带来的问题:
 - 多进程同时读写不同文件时造成对spinlock的竞争



LRU链表

Task2: buffer cache

实验内容:

- 修改xv6 block cache的实现, 减少内存申请过程中的对锁的竞争

Buffer cache in XV6: kernel/bio.c

- binit(): 初始化buffer cache (*)
- bget(): 寻找给定block的buffer cache (*)
 - 如果没找到: 新建 (0->blockno1) 或者替换 (blockno2 ->blockno1)
- brelease(): 释放buffer cache (*)
- bread(): 从block读取数据到buffer cache
- bwrite(): 将buffer cache数据写入block
- bpin/bunpin: 修改引用计数

Task2: buffer cache

实验内容:

- 修改xv6 block cache的实现, 减少内存申请过程中的对锁的竞争

实现方式:

- Per-CPU不再可行
 - Buffer cache与block是一一对应的关系, 无法预先分配到不同的CPU
- 利用hash将不同的block分成不同的组
 - 每个blockno映射到固定的hash bucket
 - 简单起见, 可以使用固定的hash bucket数量 (使用质数来尽量减少conflict)
 - 每个不同的bucket有一个自己的buf list
 - 每个bucket仍需要有一个锁来保证一一对应关系
- 当该bucket中没有空闲的buf时, 需要去别的bucket窃取空闲的buf

Task2: buffer cache

实验内容:

- 修改xv6 block cache的实现, 减少内存申请过程中的对锁的竞争

一些提示:

- 移除全局链表 bcache.head, 用hash bucket代替 (自己实现)
- 简单起见, 可以不用实现LRU, 只需要找到空闲的buf即可 (`refcnt == 0`)
- 当在当前bucket中没有空闲buffer时, 需要到其他bucket中寻找空闲的buffer
 - A simple way: 先释放所有锁, 再串行在其他bucket中加锁, 寻找
 - 注意避免死锁 (e.g. 持有当前bucket的锁, 同时获取其他的bucket的锁, 交叉等待)
- 先串行化bget()来测试是否有race conditions (通过在函数开头/结尾分别加/解锁来实现)

Task2: buffer cache



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

附加题：维护LRU List，在驱逐buffer时，将最近用的最少的块驱逐，而不是任意驱逐一个空闲块



Q&A

任何问题？